Roteiro de fala — Defesa do TCC

Autosim: DSL para o Ensino de Autômatos Finitos

Tempo total: **15 minutos** · 42 slides

Como usar: a coluna *Tempo* traz o tempo do slide e, abaixo, o tempo acumulado ao final dele. Use o acumulado para conferir o ritmo durante a fala. As falas foram dimensionadas a cerca de 140 palavras por minuto. Nos slides de *Roteiro* do início de cada seção, diga só a frase de transição e avance.

Slide	Tempo	Fala
Abertura		**0:30**
1 — Capa	**0:15** até 0:15	Boa noite a todos. Nós somos João Thomaz Vieira e Klaus Siegfried Beckmann e vamos apresentar o nosso trabalho de conclusão de curso: a Autosim, uma ferramenta baseada em linguagem de domínio específico para o ensino de autômatos finitos, orientada pelo professor Diógenes Furlan.
2 — Roteiro	**0:15** até 0:30	A apresentação segue a estrutura da monografia: introdução, a fundamentação sobre autômatos, compiladores e Rust, os trabalhos relacionados, o projeto da linguagem, a implementação, os resultados e a conclusão.
1. Introdução		**2:20**
3 — Roteiro	**0:05** até 0:35	Começando pela introdução.
4 — Contexto e motivação	**0:40** até 1:15	Autômatos finitos e linguagens formais são um dos pilares da Ciência da Computação: é onde o aluno conhece máquinas abstratas, os limites da computação e a base dos compiladores. Mesmo assim, é uma disciplina conhecida por ser difícil. Morazán e Antunez apontam o porquê: na programação, o aluno escreve, executa e vê o resultado na hora; em autômatos, ele projeta e prova a correção no papel, sem poder experimentar nem receber retorno imediato. Isso afasta o formalismo da prática.
5 — O problema	**0:35** até 1:50	Existem ferramentas para ajudar, e a mais conhecida é o JFLAP. Mas elas são exclusivamente visuais: o autômato é montado arrastando e soltando estados. Com isso, a descrição não é texto, o que dificulta versionar e compartilhar; não há validação formal da estrutura; e, quando o aluno erra, não recebe um diagnóstico preciso. A lacuna é uma ferramenta que junte o rigor da descrição textual, o diagnóstico de um compilador e a interatividade da simulação visual.
6 — A proposta: Autosim	**0:35** até 2:25	Nossa proposta é a Autosim, que acopla três partes: uma DSL textual, declarativa e em português, para descrever AFDs e AFNs; um compilador pedagógico, com análise léxica, sintática e semântica, que aponta os erros com linha e coluna; e uma interface gráfica que desenha o diagrama a partir do texto e anima a simulação passo a passo. A contribuição central está nesse acoplamento: o aluno escreve o texto e a interface é gerada automaticamente, sincronizada com ele.

Slide	Tempo	Fala
7 — Objetivos	**0:25** até 2:50	O objetivo geral é integrar descrição textual, compilação pedagógica e simulação visual em um único ambiente. Para isso, projetamos a DSL com gramática em EBNF, implementamos o compilador, desenvolvemos o simulador, construímos a interface gráfica e validamos tudo com autômatos clássicos e cenários de erro.
2. Autômatos Finitos e Linguagens Formais	**1:35**	
8 — Roteiro	**0:05** até 2:55	Agora, a fundamentação, começando pelos autômatos.
9 — Autômatos finitos	**0:35** até 3:30	Formalmente, um AFD é uma quíntupla: um conjunto finito de estados, um alfabeto, uma função de transição, um estado inicial e um conjunto de estados de aceitação. A linguagem reconhecida é formada pelas cadeias que, partindo do estado inicial, terminam em um estado final. No AFN, a transição leva a um conjunto de estados e pode haver transições vazias, as transições épsilon. Os dois têm o mesmo poder: reconhecem exatamente as linguagens regulares.
10 — Exemplo prático	**0:30** até 4:00	No diagrama, os círculos são os estados, a seta de entrada marca o inicial, o círculo duplo é o final, e as arestas são a função de transição, que também está na tabela ao lado. Esse autômato aceita as cadeias que contêm `ab`: aceita `ab` e `aab`, rejeita `ba` e a cadeia vazia. Vamos reencontrar esse mesmo exemplo na DSL e na simulação.
11 — Hierarquia de Chomsky	**0:25** até 4:25	Esses autômatos se encaixam na Hierarquia de Chomsky, que classifica as linguagens pela forma das regras das gramáticas. No tipo 3 estão as gramáticas regulares, que geram exatamente as linguagens reconhecidas pelos autômatos finitos. É nesse nível que o trabalho atua.
3. Compiladores e Linguagens de Domínio Específico	**1:00**	
12 — Roteiro	**0:05** até 4:30	Em seguida, compiladores e linguagens de domínio específico.
13 — Fases de análise	**0:30** até 5:00	Como a Autosim tem um compilador, vale lembrar as fases de análise. A léxica agrupa os caracteres em *tokens*, usando expressões regulares. A sintática organiza esses *tokens* em uma árvore sintática abstrata, a AST. E a semântica verifica o que a gramática sozinha não captura, como referências a algo que não foi declarado.
14 — DSL	**0:25** até 5:25	Uma DSL, segundo Fowler, é uma linguagem de expressividade limitada, voltada a um domínio específico: perde generalidade, mas ganha concisão e legibilidade. Ela pode ser interna, embutida em outra linguagem, ou externa, com sintaxe própria e compilador dedicado. A Autosim é uma DSL externa.
4. Rust como Linguagem de Implementação	**0:30**	
15 — Roteiro	**0:05** até 5:30	Sobre a linguagem de implementação:
16 — Por que Rust?	**0:25** até 5:55	Escolhemos Rust porque ele tem desempenho comparável a C e C++, com segurança de memória garantida na compilação e sem coletor de lixo. O sistema de tipos ajuda a representar *tokens*, nós da AST e estados sem inconsistências, e o ecossistema, com o Cargo, já oferece bibliotecas maduras para *lexing*, *parsing* e interface gráfica.

Slide	Tempo	Fala
5. Trabalhos Relacionados		**1:05**
17 — Roteiro	**0:05** até 6:00	Agora, o que já existe.
18 — Panorama	**0:30** até 6:30	Analisamos cinco trabalhos. O JFLAP é a referência mundial e cobre toda a hierarquia, mas é só gráfico. O GAM, da UFLA, é limitado a 10 estados e 3 símbolos. O simulador da UFSC roda na web, mas a construção continua puramente gráfica. O AutomatoGraph gera a gramática regular, mas a simulação é só parcialmente visual. E o trabalho de Bruzzone, em Rust, tem rigor de engenharia, mas não é voltado ao ensino.
19 — Análise comparativa	**0:30** até 7:00	Colocando lado a lado: nenhuma das ferramentas voltadas ao ensino tem descrição textual, versionamento ou compilador com diagnóstico; e a única com versionamento não é voltada ao ensino nem tem simulação visual. A Autosim é a única que reúne as cinco características. É essa lacuna que o trabalho ocupa.
6. Projeto da DSL		**1:25**
20 — Roteiro	**0:05** até 7:05	Passando para o projeto da linguagem.
21 — A linguagem: um exemplo	**0:35** até 7:40	Este é o mesmo autômato do exemplo anterior, escrito na Autosim. O arquivo `.asl` começa declarando o alfabeto. Depois vem o autômato: o tipo, AFD, o nome, os estados, o inicial, os finais e as transições, cada uma no formato origem, seta, destino, "com" símbolo. No fim, os comandos `simular` testam cadeias: `ab` é aceita e `bbb` é rejeitada. A sintaxe é em português e próxima da definição formal.
22 — Gramática (EBNF)	**0:20** até 8:00	A linguagem foi formalizada em EBNF. Ela é livre de contexto e pode ser analisada por um *parser* descendente preditivo, olhando um único símbolo à frente. São 11 palavras reservadas, e nelas maiúsculas e minúsculas não fazem diferença.
23 — Decisões de projeto	**0:25** até 8:25	O alfabeto é declarado uma vez só, o que reforça o conceito teórico e simplifica a verificação. O tipo, AFD ou AFN, é explícito, para checar determinismo só no AFD. A sintaxe é em português, para o foco ficar nos conceitos, e as palavras-chave não diferenciam maiúsculas, para não atrapalhar o aluno. E há suporte a transições épsilon com as palavras `epsilon` ou `eps`.
7. Implementação		**2:50**
24 — Roteiro	**0:05** até 8:30	Agora, a implementação.
25 — Pipeline de compilação	**0:25** até 8:55	O arquivo `.asl` passa pelo analisador léxico, feito com a biblioteca Logos, pelo sintático, com Chumsky, e pelo semântico. Se estiver tudo certo, segue para o simulador e para a interface, feita com Iced. Se alguma fase encontrar erro, a execução para ali e os diagnósticos são exibidos com a biblioteca Ariadne.

Slide	Tempo	Fala
26 — Análise léxica e sintática	**0:25** até 9:20	No léxico, o Logos gera um AFD a partir das anotações do tipo `Token`; ou seja, usamos um autômato para ler a linguagem de autômatos. Espaços e comentários são descartados, e um caractere inválido é reportado com linha e coluna. No sintático, o Chumsky usa *parser combinators*: a gramática é escrita compondo funções, e o resultado é a AST.
27 — Análise semântica	**0:30** até 9:50	A semântica tem três fases. Primeiro, a tabela de símbolos, que detecta estados, símbolos ou autômatos duplicados. Depois, a verificação de referências: estado não declarado, símbolo fora do alfabeto ou simulação de um autômato que não existe. Por último, só para AFD, o determinismo: sem épsilon e com uma única transição por estado e símbolo. Os erros são acumulados, então o aluno vê todos de uma vez.
28 — Simulador	**0:35** até 10:25	O simulador aplica a função de transição sobre a cadeia e guarda todas as configurações intermediárias. Em vez de mostrar a nuvem de estados simultâneos do AFN, ele mostra um caminho único: se a cadeia é aceita, o caminho que aceita; se é rejeitada, o caminho que chegou mais perto da aceitação. Esse caminho é escolhido por uma busca em largura. E, como o histórico fica guardado, o aluno pode avançar e voltar na simulação.
29 — Interface gráfica	**0:25** até 10:50	A interface, feita com Iced, desenha o diagrama automaticamente a partir do texto. A simulação pode ser passo a passo ou automática, a 800 milissegundos por passo, destacando o estado atual, a transição usada e o que já foi lido. Dá para arrastar e dar zoom no diagrama, e o painel fica verde quando a cadeia é aceita e vermelho quando é rejeitada.
30 — Diagnóstico de erros	**0:25** até 11:15	Cada mensagem de erro traz linha e coluna, o trecho do código com realce e uma seta no ponto do erro, com a descrição em português. Por exemplo: “token inválido”, “AFD não pode ter epsilon” ou “esperava inicial”. Isso troca o tentar e errar visual por um retorno imediato e preciso.
8. Resultados		**2:30**
31 — Roteiro	**0:05** até 11:20	Vamos aos resultados.
32 — Interface: visão geral	**0:20** até 11:40	Esta é a interface com o autômato do exemplo carregado. No topo, escolhemos a simulação, aqui com a cadeia `ab`. O diagrama foi gerado direto do arquivo; embaixo aparecem a entrada, com o próximo símbolo destacado, e os botões Voltar, Passo, Rodar, Reset e Centralizar.
33 — Simulação: aceitação	**0:25** até 12:05	Executando a simulação de `ab`: o autômato sai de q_0, lê `a` e vai para q_1, lê `b` e chega em q_2, que é final. A cadeia é aceita e o painel fica verde.
34 — Simulação: rejeição	**0:20** até 12:25	Já com `bbb`, o autômato fica em q_0, lendo `b` três vezes. A cadeia termina fora de um estado final, então é rejeitada e o painel fica vermelho. Embaixo aparece o histórico de cada passo.
35 — AFN-ε e diagramas grandes	**0:20** até 12:45	A ferramenta também trata AFNs com transições épsilon, à esquerda, e autômatos maiores, como este AFD de 14 estados, que se navega arrastando e dando zoom.

Slide	Tempo	Fala
36 — Testes automatizados	**0:20** até 13:05	Para validar, escrevemos 51 testes automatizados: 12 do léxico, 15 do sintático, 15 do semântico e 9 do simulador. Eles cobrem os *tokens*, a construção da AST e todas as verificações semânticas, e todos passaram.
37 — Autômatos clássicos	**0:20** até 13:25	Também simulamos autômatos clássicos do Hopcroft e do Sipser: AFD, AFN e AFN com épsilon, com cadeias que deveriam ser aceitas e rejeitadas, incluindo a cadeia vazia. Todos os vereditos bateram com o esperado.
38 — Cenários de erro	**0:20** até 13:45	E criamos arquivos com erros propositais, como um aluno faria: erros semânticos, AFD com épsilon, AFD não determinístico, palavra-chave faltando e caractere inválido. Em todos, a mensagem saiu com linha, coluna, trecho realçado e descrição em português.
9. Conclusão		**1:15**
39 — Roteiro	**0:05** até 13:50	Para concluir.
40 — Contribuições	**0:30** até 14:20	Em resumo, a Autosim aproxima a descrição formal da visualização. O trabalho entrega uma DSL em português para AFDs e AFNs, formalizada em EBNF; um compilador pedagógico completo, com diagnósticos localizados; um simulador com histórico e navegação nos dois sentidos; e uma interface que anima a simulação. Tudo validado com 51 testes e simulações clássicas, todos corretos.
41 — Limitações e trabalhos futuros	**0:30** até 14:50	Como limitações: um único alfabeto por arquivo, não há modo de linha de comando, é um arquivo por execução, não há editor embutido nem conversão de AFN para AFD ou minimização. Como trabalhos futuros, queremos tratar esses pontos, incluir expressões regulares e gramáticas, estender a ferramenta para autômatos com pilha e máquinas de Turing e, principalmente, fazer estudos com estudantes para medir o impacto no ensino.
42 — Obrigado	**0:10** até 15:00	Obrigado pela atenção. Ficamos à disposição para as perguntas da banca.